Document Number:	p3456r0
Date:	2024-10-15
Target:	SG1
Revises:	
Reply to:	Gor Nishanov (gorn@microsoft.com)

system_scheduler on Win32, Darwin and Linux

Abstract

This document compares the features offered by the global threadpool on Windows and MacOS with the facilities proposed in the P2079R4 System Execution Context paper. It explores extensions to P2079R4 that capture most of the primitives offered by the operating systems and outlines implementations for Windows, MacOS, and Linux.

Overview

Win32 and Darwin offer global system threadpools with the following features:

1. Post work items for execution.
2. Schedule a work item to run at a particular time (or after a delay).
3. Associate an I/O operation with a threadpool, so that the handler processing completion will be executed by the threadpool.
4. Bulk execution (Darwin only), though a bulk algorithm running over post work item results in better performance (see bulk section).
5. Allow associating priorities with work items.
6. Maintain an optimal number of threads processing work items:
    1. If a thread gets blocked, another one is released/created to maintain the desired number of active threads.
    2. The number of active threads is kept proportional to the number of cores.
    3. Guards against thread explosion (when newly created threads get blocked).
    4. Shrinks the number of threadpool threads to zero when not needed.

The following table summarizes the features:

	p2079r4	Windows	MacOS	Asio
schedule	+	+	+	+
schedule_at		+	+	+

	p2079r4	Windows	MacOS	Asio
defer				+
bulk	+		+	
i/o		+	+	+
priorities		+	+	

In subsequent sections we will try to outline whether we can get as much parity with system threadpools with C++26 system context and what can be added later.

system_scheduler of P2079r4

As a starting point, we will use p2079r4's system_scheduler and over the course of this paper expand it to cover most of the functionality offered by operation system global threadpools and boost asio.

```
system_scheduler get_system_scheduler();

class system_scheduler() {
public:
  system_scheduler() = delete;
  bool operator==(const system_scheduler&) const noexcept;
  std::execution::forward_progress_guarantee get_forward_progress_guar

  sender auto schedule();
  sender auto bulk(integral auto i, auto f);
};
```

Priorities

Win32 threadpool offers three priority levels: high, normal and low. When a thread comes to pick up work, it will only pick up work from a lower priority queue, only when there is no work items in the higher priority queues. MacOS offers additional priority level `background` that is of lower priority than `low` and all the work with `background` priority will not run when low-power mode is enabled.

To reach parity we recommend to to extend `get_system_scheduler()` to take an optional `priority` parameter to take advantage of the underlying system threadpools. On systems without OS threadpool, this can be easily supported by providing a queue per priority level.

```
enum class system_scheduler_priority {
   background
   low,
   normal,
   high,
};

system_scheduler get_system_scheduler(system_scheduler_priority priori
```

On Darwin, it can be implemented as:

```
class system_scheduler {
   ...
   using native_handle_type = dispatch_queue_t;
   native_handle_type native_handle();

   friend system_scheduler get_system_scheduler();
private:
   explicit system_scheduler(native_handle_type h) : handle{h}
   native_handle_type handle{};
};

system_scheduler get_system_scheduler(system_scheduler_priority priori
   static array<int, 4> arr = {
      DISPATCH_QUEUE_PRIORITY_BACKGROUND,
      DISPATCH_QUEUE_PRIORITY_LOW,
      DISPATCH_QUEUE_PRIORITY_DEFAULT,
      DISPATCH_QUEUE_PRIORITY_HIGH
   };
   auto queue = dispatch_get_global_queue(to_underlying(priority), 0)
   return system_scheduler(q);
}
```

On Windows:

```
class system_scheduler {
   ...
   using native_handle_type = TP_CALLBACK_ENVIRON*;
   native_handle_type native_handle();

   friend system_scheduler get_system_scheduler();
private:
   explicit system_scheduler(native_handle_type h) : handle{h}
   native_handle_type handle{};
};

TP_CALLBACK_ENVIRON env_for_priority(TP_CALLBACK_PRIORITY pri) {
  TP_CALLBACK_ENVIRON result;
  InitializeThreadpoolEnvironment(&result);
  SetThreadpoolCallbackPriority(&result, pri);
  return result;
}
array<TP_CALLBACK_ENVIRON, 4> arr = {
   env_for_priority(TP_CALLBACK_PRIORITY_LOW),
   env_for_priority(TP_CALLBACK_PRIORITY_LOW),
   env_for_priority(TP_CALLBACK_PRIORITY_NORMAL),
   env_for_priority(TP_CALLBACK_PRIORITY_HIGH),
};

system_scheduler get_system_scheduler(system_scheduler_priority priori
{
   return system_scheduler(arr[to_underlying(priority)]);
}
```

One design point is whether normal should be a default enum value, as in `system_scheduler_priority{} == system_scheduler_priority::normal`.

If we choose to do that, the enum will look like:

```
enum class system_scheduler_priority : int {
   background = -2,
   low = -1,
   normal = 0,
   high = 1,
};
```

Bulk

Bulk returns a sender describing the task of invoking the provided function with every index in the provided shape along with the values sent by the input sender. For example, the following coroutine will perform matrix multiplication using bulk.

```
task<void> matmul(float* xout, float* x, float* w, int n, int d) {
   auto sh = get_system_scheduler();
   // Parallel matrix multiplication using bulk.
   // W (d,n) * x (n) -> xout (d)
   co_await sh.bulk(d, [&](size_t i) {
        float val = 0.0f;
        for (int j = 0; j < n; j++)
            val += w[i * n + j] * x[j];

        xout[i] = val;
   });
}
```

Darwin libdispatch offers the dispatch_apply_f that submits a single function to the dispatch queue and causes the function to be executed the specified number of times.

```
void dispatch_apply_f(size_t iterations, dispatch_queue_t queue, void
```

Windows does not have an equivalent, but, such facility can be readily implemented as an algorithm on top of non-bulk schedule.

In a few tests we ran, on Darwin, running bulk algorithm over scheduler is more performant than the dispatch_apply_f.

Additionally bulk algorithm supports cancellation that can prompt early exit from the calculation when not needed.

Another observation is that on MSVC, running parallel algorithms (which internally use the global threadpool) completely starves all other threadpool work until the parallel computation is completed. This appears to be suboptimal behavior.

We would like to propose that bulk should:

1. support cancellation
2. cooperate with other items in the threadpool to not starve "concurrency" work items while "parallel" computation is run.

Defer

This feature is not available in Win32 or Darwin and only offered by by boost::asio and earlier revisions of executor proposal.

It allows to defer an execution of a work item immediately after the current work item returns control to the threadpool.

- it allows to avoid stack overflow when performing inline completion without the need to resort to posting a threadpool work item
- it allows to avoid deadlocks when code executing a callback is holding a lock and some actions need to be deferred until the control is returned to a threadpool.

The implementation is straightforward. A scheduler will need to maintain a thread-local queue of work that will accumulate deferred items and execute them all immediately after the current executing work item finishes.

```
system_scheduler get_system_scheduler();

class system_scheduler() {
public:
  system_scheduler() = delete;
  bool operator==(const system_scheduler&) const noexcept;
  std::execution::forward_progress_guarantee get_forward_progress_guar

  sender auto schedule();
  sender auto defer(); // <-- freshly added
  sender auto bulk(integral auto i, auto f);
};
```

Timed

Win32, Darwin and Asio threadpools support scheduling work items based on time (absolute or relative).

Underlying implementation maintains a data structure to keep the scheduled work items in the order of expiration. Usually, it is a pairing heap (O(1) insert, O(log n) extract expiring item).

To reduce impact of O(log n), Windows gives an opportunity to the user to indicate whether it is OK to delay execution of a work item with a specified window, thus allowing to group some of the timed items together, reducing the total number of elements maintained by the heap.

Windows also support cancellation of timed work items, whereas Darwin does not.

Thus on Darwin, support for timed operations will need to rely on C++ runtime maintained pairing heap if we would like to support cancellation, since now the heap is in the c++ library implementor hand's, we can also add support for coalescing window increasing performance when number of timed items is high.

```
template <typename Rep, typename Ratio>
schedule_after_sender system_scheduler::schedule_after(std::chrono::du

template <typename Clock, typename Duration>
schedule_at_sender system_scheduler::schedule_at(std::chrono::time_poi
```

We choose to keep the window in milliseconds as it matches Win32 API. While it is possible to have it specified in microseconds or nanoseconds (or have it as templatized duration), we ended up with milliseconds based on simplicity and windows experience.

Elastic threadpool

One fundamental feature of OS provided threadpools is that they have visibility in what threads are doing and can make intelligent decisions when to remove or add a thread to the threadpool.

We explored two contending approaches:

1. Use Berkeley Packet Filter to inject code into the kernel to report when thread goes to sleep or wakes up
2. Do periodic sampling of all threadpool threads by reading /proc/self/task/{}/stat

The first approach seems sub-optimal as it forces system wide overhead for all processes and typically requires root privilege to install.

Sampling of /proc/self/task/{}/stat seems more promising. This is the approach used by libdispatch on Linux. Moreover, the overhead of sampling can be reduced significantly utilizing `io_uring` to sample thousands of thread stats in a one kernel transition.

Knowledge of the threadpool thread state allows to implement the desired policies to grow and shrink threadpool as needed.

Scheduled item cancellation

One open question is what to do when a pending work item (via schedule() or as a result of I/O completion) is cancelled. We can see several strategies.

1. (Conservative) Leave the work item in the queue, it will be processed like a non-cancelled work item, with the exception that instead of calling set_value, set_stopped will be called.
2. (Eager) Remove work item from the queue. Execute the handler inline.
3. (Reprioritize) Move the cancelled work items to the beginning of the queue
4. (Defer) Move the cancelled work item to the deferred queue

Out of this options, the first one looks least problematic, while it does not speed up completion of the cancelled item if there are many items ahead in the queue, it does not disturb the

execution of other items and if there are many cancellations, they will be handled by threadpool concurrently.

Eager, can lead to deadlocks, stack overflows, slowing down cancellation as it would be executed by the thread initiating the cancellation.

Defer is immune from the deadlock, but, has the same issue of running the cancellation on the same thread.

Finally, eager, prioritizes cancelled items over regular items. It may or many not be beneficial over the conservative approach depending on the exact scenario.

Thus, conservative approach is recommended as a default.

Timer implementation

The most likely implementation of a timer (in the cases when such facility is not available in the OS) would be to maintain a heap of timer items in the order of expiration and use a threadpool control thread or other facilities to wait until the nearest timer expires. Once expired, the work item is queued into a threadpool queue as if it was scheduled via schedule_sender.

If a timed work is cancelled prior to expiration, it is also posted into a threadpool in a cancelled state.

I/O

A libunifex explored how sender/receive I/O may look like. For example:

```
task<void> read(auto scheduler sched) {
   auto in = open_file_read_only(sched, "file.txt");
   std::array<char, 1024> buffer;
   co_await async_read_some_at(
      in, 0, as_writable_bytes(span{buffer.data(), buffer.size()}));
}
```

While it is unlikely that in time for C++26 we will be able to specify I/O suite covering various aspect of the I/O, we can expose `native_handle()` to the `system_scheduler` that would allow developers to write libraries on top providing an I/O support that will be executing completions on system scheduler.

Early in the priority section we already got a sneak preview of native_handle. In this section we can show how it can be used to support an I/O.

In Windows, native_handle is an alias to `TP_CALLBACK_ENVIRON*` that carries information about what threadpool to use and what is the priorities of the items being scheduled. It can be

used with existing win32 threadpool APIs as follows:

```
auto io = CreateThreadpoolIo(file, cb, context,
   get_system_scheduler().native_handle());
```

Of course, we expect that the users will have a nice C++ wrappers around raw win32 APIs, but, the purpose of this to show how to make C++ system context interact with components that expect windows threadpool (TP_CALLBACK_ENV) as an argument.

Similarly, on Darwin, it would look

```
dispatch_read(f, 1024, get_system_scheduler().native_handle(),
^(dispatch_data_t data, int error) {...});
```

It is likely that Linux implementation will use io_uring to send commands and receive completions and subsequently it will post completion handle to the threadpool as if by schedule(). Thus, on Linux, native handle needs to have enough context for the user to be able to develop their own I/O wrappers on top.

Summary

Thus, in order to expose most of the features of underlying OS global threadpool the system_context would look like:

```
enum class system_scheduler_priority {
   background
   low,
   normal,
   high,
};

system_scheduler get_system_scheduler(system_scheduler_priority priori

class system_scheduler() {
public:
  system_scheduler() = delete;
  bool operator==(const system_scheduler&) const noexcept;
  std::execution::forward_progress_guarantee get_forward_progress_guar

  using native_handle_type = implementation-defined;
  native_handle_type native_handle();

  sender auto schedule();
  sender auto defer();
  sender auto bulk(integral auto i, auto f);

  template <typename Rep, typename Ratio>
  schedule_after_sender system_scheduler::schedule_after(
     std::chrono::duration<Rep, Ratio> delay,
     milliseconds window = {}) const noexcept;

  template <typename Clock, typename Duration>
  schedule_at_sender system_scheduler::schedule_at(
     std::chrono::time_point<Clock, Duration>& abs_time,
     milliseconds window = {}) const noexcept;
};
```

This paper was coming in hot for the deadline. We intend to clean it up and improved before seeing it in Poland.

References:

[p2300] https://wg21.link/p2300 std::execution

[p2079r4] https://wg21.link/p2079r4 System execution context

[asio-executors] https://github.com/chriskohlhoff/executors/

[win32 threadpool details] https://www.youtube.com/watch?v=CzgNVuXVMWo

[libdispatch] https://developer.apple.com/documentation/dispatch?language=objc

[io_uring] https://kernel.dk/io_uring.pdf